

JIRA Task 05 — Swagger / OpenAPI & Professional API Documentation

Microservices Architecture Project — implementation, verification and final review guide

Objective: Create complete professional API documentation that another developer can use without reading the source code.

1. OpenAPI and Swagger — brief explanation

- **OpenAPI:** a standard way to describe a REST API — endpoints, parameters, request bodies, responses and errors.
- **Swagger UI:** a browser interface that displays the OpenAPI description and lets developers test endpoints.
- **API specification:** the API contract describing what clients can send and what the service returns.
- **Request schema:** the structure, fields and constraints of incoming data.
- **Response schema:** the structure and meaning of data returned by the service.

2. How JIRA Task 05 was completed

Step 1 — Understand OpenAPI — Learn the core OpenAPI/Swagger concepts and why API documentation is useful.

Step 2 — Add OpenAPI dependency — Add springdoc OpenAPI support to the Spring Boot service so OpenAPI JSON and Swagger UI are generated.

Step 3 — Document every endpoint — Document the Product Service v1 GET, POST, PUT, PATCH and DELETE endpoints.

Step 4 — Document request parameters — Describe path parameters, query parameters, request bodies, examples and required/optional fields.

Step 5 — Document responses — Describe successful and error status codes, response schemas and realistic examples.

3. Product Service API documented

Method	Endpoint	Purpose
GET	/api/v1/products	List products with pagination, sorting and filtering.
GET	/api/v1/products/{id}	Get one product by ID.
POST	/api/v1/products	Create a product.
PUT	/api/v1/products/{id}	Update a product.
PATCH	/api/v1/products/{id}	Partially update a product.
DELETE	/api/v1/products/{id}	Delete a product.

4. Request parameters and DTO documentation

- **Path parameters:** Product ID is described as required and has an example such as 101.
- **Query parameters:** page, size, sort, category, minPrice and maxPrice are documented with descriptions and examples.
- **Request DTOs:** CreateProductRequest, UpdateProductRequest and PatchProductRequest have OpenAPI field descriptions, examples and validation metadata.
- **Response DTO:** ProductResponse documents id, name, description, price, category, stock, status, createdAt and updatedAt.
- **Error DTO:** ErrorResponse documents status, message and timestamp so error responses are understandable.
- **Validation note:** the current PATCH controller does not use `@Valid`; its validation annotations should not be assumed to execute unless runtime behavior is intentionally changed.

5. Response documentation

Code	Documented meaning
200	Successful retrieval or update.
201	Product created successfully.
204	Product deleted successfully; no response body.
400	Invalid product data or invalid list parameters.
404	Product not found.
409	Add only if the actual application has a conflict scenario/handler.
500	Add when the service's exception handling can return an internal server error.

6. Screenshot evidence from the completed work

The supplied screenshot is included below as evidence of the final API review checklist. It shows Step 10 — Final API review and the items to check: naming standards, HTTP methods, status codes, DTOs, validation, error handling, pagination, sorting, filtering, API versioning and Swagger documentation.

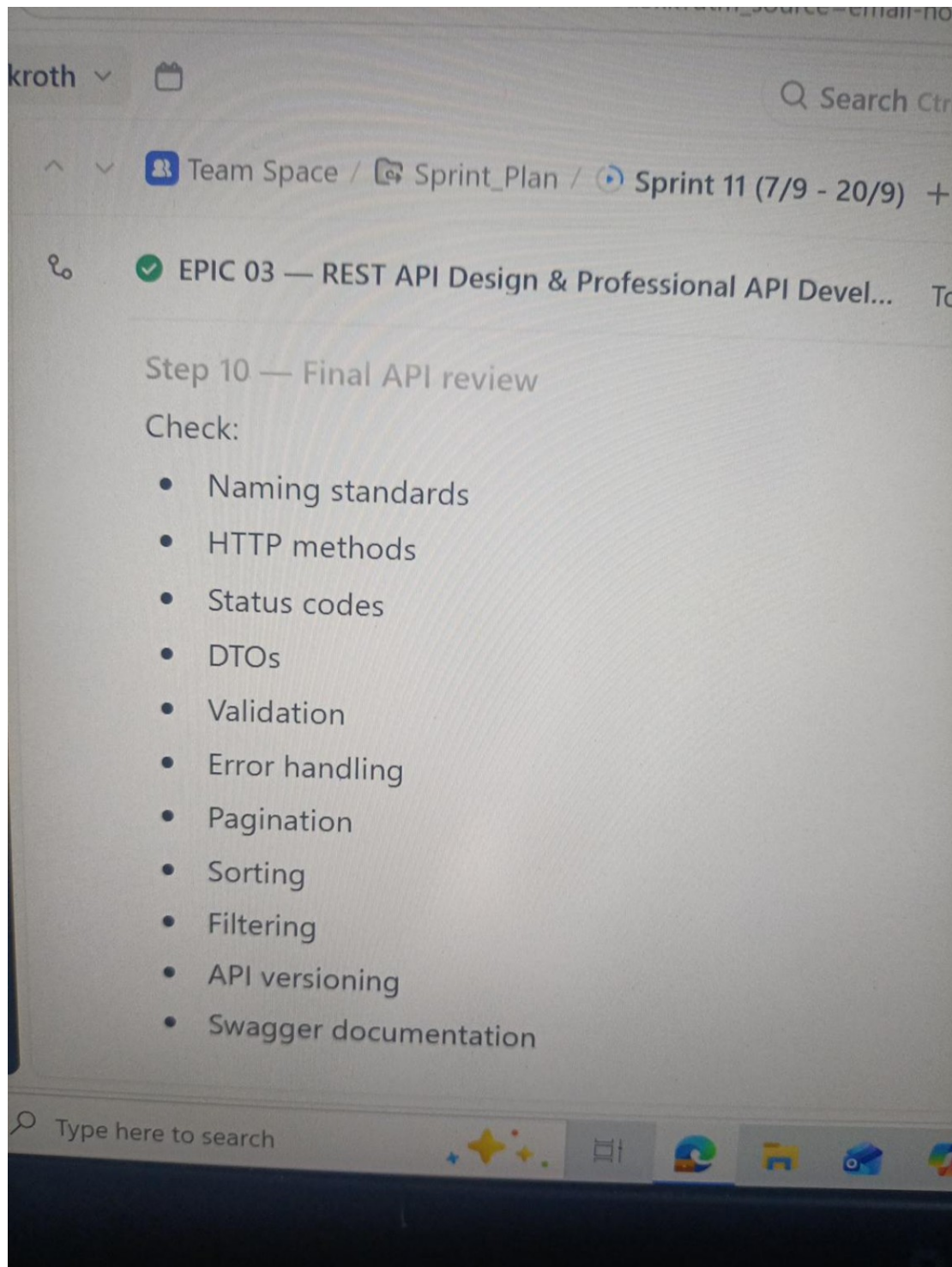


Figure 1 — Supplied project-board screenshot for Step 10: Final API review.

7. Swagger UI evidence from the Product Service

- **POST /api/v1/products:** Swagger UI shows 201 Product created successfully with application/json and a full ProductResponse example.
- **POST validation:** Swagger UI shows 400 Invalid product data with an ErrorResponse example.
- **GET /api/v1/products/{id}:** shows 200 Product found successfully and 404 Product not found, with meaningful examples.

- **GET /api/v1/products:** shows a paginated response containing content, page information, size, sorting and total counts.
- **PUT and PATCH:** successful 200 responses show the updated product.
- **API versioning:** the documented Product Service endpoints use /api/v1; the separate v2 controller shown in Swagger should be treated as a separate API version.

8. Step 10 — Final API review checklist

Check	What it means
Naming standards	Consistent and understandable resource/endpoint names.
HTTP methods	GET, POST, PUT, PATCH and DELETE are used correctly.
Status codes	Codes accurately match actual API behavior.
DTOs	Request/response models are clearly documented.
Validation	Required fields and constraints are documented and runtime behavior is verified.
Error handling	Exception handling is reflected accurately in Swagger.
Pagination	Page and size behavior is clear.
Sorting	Sort field and direction are documented.
Filtering	Category and price filters are documented.
API versioning	Versioned paths such as /api/v1 are clear.
Swagger documentation	Swagger UI loads and every intended endpoint is useful to another developer.

9. How to run and verify the documentation

Compile the service:

```
mvn clean compile
```

Run the service:

```
mvn spring-boot:run
```

For the Product Service, Swagger UI was verified at <http://localhost:8082/swagger-ui/index.html>. Open the Product endpoints, use **Try it out**, and compare the actual response with the documented response.

10. Completing the whole microservices task

The Product Service is the working example. For full project completion, apply the same approach to Order Service and User Service: add the compatible springdoc dependency, configure OpenAPI, document every endpoint, document parameters and DTOs, and document the actual success/error responses. Dependency versions must match each service's Spring Boot version.

Important: do not invent 409 or 500 responses simply because they appear in a generic checklist. Document a status code when the service can actually return it. Professional API documentation should describe real application behavior.

11. Final result

The Product Service portion of JIRA Task 05 now demonstrates professional OpenAPI/Swagger documentation: endpoints, parameters, DTOs, examples, response schemas, successful status codes and documented errors are visible in Swagger UI. The supplied screenshot is included as evidence of the Step 10 final-review checklist. The same standards should be applied to the remaining services before the overall task is marked complete.